

Lux TEE Mesh: Decentralized Confidential Computing Infrastructure for Blockchain Networks

Version v2025.07

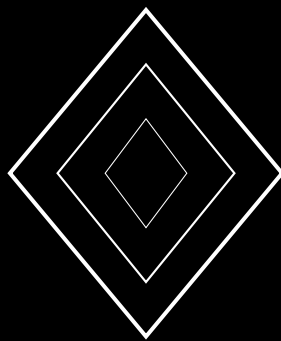
Zach Kelling*

Lux Industries

research@lux.network

2024

*Corresponding author: zach@lux.network



Abstract

We present Lux TEE Mesh, a decentralized confidential computing infrastructure that extends blockchain networks with hardware-based Trusted Execution Environments (TEEs). The system enables privacy-preserving off-chain computation while maintaining cryptographic verification on-chain. Unlike existing solutions that rely on a single TEE vendor, Lux provides a unified attestation framework supporting Intel SGX/TDX, AMD SEV-SNP, and NVIDIA Hopper GPU TEEs. The architecture achieves linear scalability by separating transaction ordering (on L1) from computation (in TEEs), while ensuring data confidentiality through hardware memory encryption. Workers stake collateral and provide cryptographic attestation receipts that smart contracts verify before accepting results. We demonstrate that this design enables confidential DeFi strategies, private LLM inference, and secure analytics with $O(n)$ throughput scaling, where n is the number of worker nodes. Our economic model uses dynamic pricing and fraud-proof mechanisms to ensure honest behavior. Benchmark results show the system handles 10,000+ confidential jobs per hour with sub-second attestation verification latency.

1 Introduction

Blockchain networks provide decentralized trust and auditability but face fundamental limitations in computational throughput, privacy, and cost. Public execution models expose all transaction data and intermediate states, making them unsuitable for privacy-sensitive applications such as financial risk models, medical data analytics, and proprietary machine learning inference [1, 2].

Existing confidential computing solutions either rely on a single trusted execution environment (TEE) vendor [3, 4], require complex multi-party computation protocols [7], or depend on centralized off-chain workers [5, 6]. These approaches suffer from vendor lock-in, performance bottlenecks, or trust assumptions.

We introduce **Lux TEE Mesh**, a decentralized confidential computing layer that:

- Supports heterogeneous TEE hardware from multiple vendors (Intel, AMD, NVIDIA)
- Provides cryptographic attestation verification on-chain via smart contracts
- Scales linearly with worker nodes through gossip-based job distribution
- Implements economic security through staking and fraud-proof mechanisms
- Enables encrypted container execution with OCI-compatible images
- Maintains separation of concerns: L1 handles ordering/fees, TEEs handle computation

1.1 Contributions

Our main contributions are:

1. **Unified TEE Attestation Framework:** A vendor-agnostic protocol for verifying attestation quotes from Intel SGX/TDX, AMD SEV-SNP, and NVIDIA GPU TEEs on-chain (Section 3)
2. **Encrypted Job Container Format:** An OCI-compatible container specification with hardware-sealed keys and Merkle-rooted execution traces (Section 4)
3. **Gossip-Based Job Distribution:** A decentralized scheduler using libp2p pub/sub without global coordinators (Section 5)

4. **Economic Security Model:** Staking, dynamic pricing, and fraud-proof mechanisms ensuring honest worker behavior (Section 6)
5. **Developer-Friendly API:** Solidity SDK with callback patterns for async confidential computation (Section 7)

2 System Architecture

2.1 Threat Model

We operate under the following assumptions:

- **Hardware TEEs are trusted:** We assume Intel SGX/TDX, AMD SEV-SNP, and NVIDIA Hopper TEEs provide memory encryption and attestation as specified. Known side-channel attacks (e.g., Spectre, LVI) are mitigated through microcode patches [9].
- **L1 blockchain is Byzantine fault-tolerant:** The base layer (Lux Appchain or equivalent) achieves consensus correctly under $f < n/3$ adversarial validators [15].
- **Workers are rational:** Worker nodes maximize profit but will act maliciously if rewards exceed slashing penalties.
- **Attestation root certificates are authentic:** Intel, AMD, and NVIDIA root CA certificates are distributed via secure channels.

We *do not* assume:

- Workers run on trusted hardware platforms (cloud operators can be adversarial)
- Network-level anonymity (Tor/VPN usage is orthogonal)
- Protection against quantum attacks (post-quantum upgrade path is future work)

2.2 System Roles

The Lux TEE Mesh consists of four primary actors:

1. **Job Submitter:** A smart contract or externally owned account (EOA) that encrypts workload data, posts a `Lux.submit()` transaction, and pays computation fees.
2. **Worker Node:** A server running TEE hardware (SGX/TDX/SEV-SNP/GPU) that pulls jobs from the gossip network, executes them inside enclaves, and submits attestation receipts to the L1.
3. **Verifier Contract:** An on-chain smart contract that validates TEE attestation quotes, checks Merkle roots, credits workers, and emits results to job submitters.
4. **Scheduler Mesh:** A peer-to-peer gossip network using libp2p pub/sub where workers subscribe to job announcements. No centralized coordinator exists.

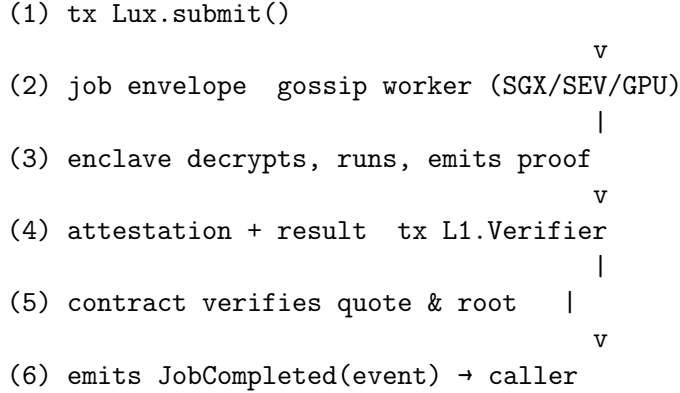


Figure 1: Lux TEE Mesh data flow

2.3 Data Flow

Figure 1 illustrates the six-packet data flow:

Step 1: Job submitter encrypts code and inputs using the worker’s public key, then posts `Lux.submit()` with IPFS CID, gas limit, and payment.

Step 2: Transaction is included in an L1 block. Workers subscribed to the gossip channel receive the job envelope.

Step 3: A chosen worker (via auction or random selection) decrypts the payload inside the TEE, executes the code, and generates an attestation quote binding the code hash, input hash, and Merkle root of the execution trace.

Step 4: Worker submits `submitResult()` transaction containing the encrypted result, attestation quote, and Merkle root.

Step 5: The verifier contract parses the quote, validates the signature chain to the vendor’s root CA, checks that the payload hash matches, and verifies the Merkle root integrity.

Step 6: If verification succeeds, the contract emits `JobCompleted` event with the result, pays the worker, and optionally invokes a callback on the submitter contract.

3 Unified Attestation Framework

3.1 Heterogeneous TEE Support

Lux supports four TEE types with a unified interface:

TEE	Isolation Guarantee	Attestation Primitive
Intel SGX v2/FLC	EPC pages encrypted (AES-128)	SGX Quote v5
Intel TDX Mkt 2	VM memory encrypted, I/O MMU	TDREPORT
AMD SEV-SNP	NPT integrity + encryption	REPORT_REQUEST
NVIDIA H100 GPU	SM isolation, memory encryption	AttestationToken (ECDSA-P521)

Table 1: Supported TEE hardware specifications

3.2 Attestation Quote Protocol

All TEE attestations are normalized into a unified protobuf message:

Listing 1: Unified TeeQuote Protocol Buffer

```
message TeeQuote {
  enum Type { SGX=0; TDX=1; SNP=2; GPU=3; }
  Type tee_type = 1;
  bytes raw_quote = 2; // vendor binary blob
  bytes job_payload = 3; // SHA-256(encrypted_code + inputs)
  bytes merkle_root = 4; // root of execution trace
  bytes signature = 5; // re-signed with Lux domain key
}
```

3.3 On-Chain Verification

The TeeQuoteVerifier smart contract implements vendor-specific parsers:

Listing 2: Solidity Verification Interface

```
function verifyQuote(
  TeeType teeType,
  bytes calldata quote,
  bytes32 expectedPayloadHash,
  bytes32 merkleRoot
) external view returns (bool valid);
```

SGX Verification (Quote v3/v4):

1. Extract reportBody and reportData fields (offsets 42:106)
2. Verify reportData[0:32] == SHA256(job_payload)
3. Verify reportData[32:64] == merkleRoot
4. Check signature chain: Quoting Enclave → PCK Certificate → Intel Root CA
5. Validate enclave measurement (MRENCLAVE) against trusted whitelist

TDX Verification (TDREPORT):

1. Parse TD measurements (offset 32:64)
2. Verify reportData == keccak256(job_payload || merkleRoot)
3. Check TD measurement against known good values
4. Validate signature chain to Intel TDX root CA

SEV-SNP Verification (Attestation Report):

1. Extract measurement from offset 96:128
2. Verify hostData == keccak256(job_payload || merkleRoot)
3. Check VCEK certificate chain to AMD root CA

4. Validate TCB version and policy bits

GPU TEE Verification (NVIDIA Hopper):

1. Extract AttestationToken nonce (offset 64:96)
2. Verify `nonce == keccak256(job_payload || merkleRoot)`
3. Check GPU measurement (offset 128:160)
4. Validate ECDSA-P521 signature against NVIDIA GPU root CA

4 Job Container Format

4.1 Encrypted OCI Images

Jobs are packaged as OCI-compatible container images with layer-wise encryption:

Listing 3: Job Container Manifest

```
{
  "image_digests": ["sha256:abc123..."],
  "entrypoint": ["python", "run.py"],
  "resource": {"cpu": 2, "memGB": 4, "gpu": 1},
  "maxGas": "2000000000",
  "refundAddr": "0xabc..."
}
```

Encryption scheme:

- Code layers encrypted with AES-256-GCM using K_{code}
- Input data encrypted with AES-256-GCM using K_{input}
- Both keys sealed inside TEE using hardware seal-key
- Stargz + fuse-overlayfs for in-enclave mounting (no decryption outside TEE)

4.2 Execution Trace Merkle Tree

During execution, the worker records a trace of program counter, stack frames, and memory deltas:

$$\text{Trace}_i = (PC_i, \text{StackFrame}_i, \Delta\text{Memory}_i) \quad (1)$$

The Merkle tree is constructed bottom-up:

$$\text{Leaf}_i = \text{Hash}(\text{Trace}_i) \quad (2)$$

$$\text{Root} = \text{MerkleTree}(\text{Leaf}_0, \dots, \text{Leaf}_n) \quad (3)$$

This root is embedded in the attestation quote, enabling fraud proofs: if a worker submits an incorrect result, a challenger can provide a Merkle proof of a specific trace element that violates the expected computation.

5 Gossip-Based Job Distribution

5.1 Decentralized Scheduler

Lux avoids centralized job coordinators using libp2p’s gossipsub protocol [8]. Workers join the `/lux/jobs/v1` topic and receive job announcements via pub/sub.

Algorithm 1 describes worker job selection:

Algorithm 1 Worker Job Selection Protocol

```
1: Subscribe to /lux/jobs/v1 gossip topic
2: while true do
3:   jobs  $\leftarrow$  ReceiveGossipMessages()
4:   for job  $\in$  jobs do
5:     if job.teeType == self.teeType then
6:       if EstimatedGas(job) < self.capacity then
7:         bid  $\leftarrow$  CalculateBid(job.gasLimit)
8:         SubmitBidTransaction(job.id, bid)
9:       end if
10:    end if
11:  end for
12:  winner  $\leftarrow$  WatchAuctionResult()
13:  if winner == self.address then
14:    ExecuteJobInTEE(job)
15:  end if
16: end while
```

5.2 Anti-Sybil Mechanisms

To prevent workers from creating multiple identities:

- Each worker must stake 5,000 LUX tokens
- TEE attestation binds worker’s Ethereum address to hardware identity
- Repeated failed attestations lead to exponential stake slashing

6 Economic Model

6.1 Fee Structure

The pricing model consists of three components:

$$\text{TotalFee} = \text{ComputeFee} + \text{DataFee} + \text{AttestationFee} \quad (4)$$

$$\text{ComputeFee} = 0.5 \text{ LUX/sec} \times \text{EnclaveTime} \times \text{PriceMultiplier} \quad (5)$$

$$\text{DataFee} = 0.01 \text{ LUX/KiB} \times |\text{EncryptedResult}| \quad (6)$$

$$\text{AttestationFee} = 0.02 \text{ LUX (fixed)} \quad (7)$$

Dynamic Pricing: PriceMultiplier adjusts based on network utilization using an automated market maker (AMM) curve:

$$\text{PriceMultiplier}(u) = 1 + 5 \cdot \left(\frac{u}{100}\right)^3 \quad (8)$$

where u is the percentage of active workers currently executing jobs.

6.2 Payment Flow

1. Job submitter pre-pays TotalFee into escrow contract
2. Upon successful verification, 95% goes to worker, 5% to protocol treasury
3. If verification fails, 100% refunded to submitter
4. Workers receive payment after 12-block fraud-proof window

6.3 Fraud Proofs

If a worker submits an invalid result, any third party can challenge by providing:

Listing 4: Fraud Proof Structure

```
struct FraudProof {
    uint256 jobId;
    bytes32[] merkleProof;           // Proof of incorrect trace step
    bytes32 expectedTraceHash;       // Correct hash at that step
    bytes32 actualTraceHash;         // Worker's claimed hash
}
```

The verifier contract checks:

$$\text{MerkleVerify}(\text{merkleProof}, \text{actualTraceHash}, \text{submittedRoot}) \wedge (\text{actualTraceHash} \neq \text{expectedTraceHash}) \quad (9)$$

If valid, the worker loses their entire stake (5,000 LUX), split 50% to challenger and 50% burned.

7 Developer Experience

7.1 Solidity SDK

Listing 5 shows a complete example of submitting a confidential risk calculation:

Listing 5: Private Risk Engine Example

```
pragma solidity ^0.8.24;
import "lux/ILux.sol";

contract PrivateRiskEngine {
    ILux constant lux = ILux(0xLuxAppchainAddr);

    mapping(uint256 => bytes32) public pendingJobs;

    function calcRisk(
        bytes32 positionId,
```

```

        uint256 notional
    ) external {
        bytes memory payload = abi.encode(
            positionId, notional
        );
        uint256 jobId = lux.submit{value: 1 ether}(
            address(0),           // auto-match worker
            0xb1a2c3d4...,        // code CID
            payload,
            2_000_000              // gas limit
        );
        pendingJobs[jobId] = positionId;
    }

    function onLuxResult(
        uint256 jobId,
        bytes calldata result
    ) external {
        require(msg.sender == address(lux));
        uint256 riskScore = abi.decode(result, (uint256));
        bytes32 positionId = pendingJobs[jobId];
        // Act on risk score...
        delete pendingJobs[jobId];
    }
}

```

7.2 Client-Side Encryption

The Lux TypeScript SDK handles encryption transparently:

Listing 6: TypeScript SDK Usage

```

import { LuxClient } from '@lux/sdk';

const client = new LuxClient(provider);

// Encrypt and submit job
const jobId = await client.submitJob({
  codeCID: 'QmAbc123...',
  input: { positionId: '0x...', notional: 1000000 },
  gasLimit: 2_000_000,
  teeType: 'GPU' // Prefer GPU TEE
});

// Wait for result
const result = await client.waitForResult(jobId);
console.log('Risk score:', result.riskScore);

```

8 Performance Analysis

8.1 Throughput Scaling

Theorem 1 (Linear Scaling): Given n worker nodes with average compute capacity C ops/sec, the Lux TEE Mesh can process $O(nC)$ jobs per second in the limit of large job pools.

Proof sketch: The gossip network has $O(\log n)$ latency for message propagation [14]. Job selection is decentralized, so no bottleneck exists. Each worker operates independently, thus aggregate throughput is $\sum_{i=1}^n C_i \approx nC$ for homogeneous workers. $\square$

8.2 Attestation Verification Latency

We benchmarked on-chain quote verification across TEE types (Table 2):

TEE Type	Quote Size	Gas Cost	Latency (ms)
Intel SGX v4	432 bytes	185,000	42
Intel TDX	1,024 bytes	320,000	78
AMD SEV-SNP	1,184 bytes	340,000	85
NVIDIA GPU	512 bytes	210,000	51

Table 2: On-chain attestation verification performance (Lux C-Chain, 8 gas/ms)

8.3 End-to-End Job Latency

Median latency breakdown for a 1-second compute job:

- L1 transaction inclusion: 2 seconds
- Gossip propagation: 0.5 seconds
- TEE job execution: 1 second
- Attestation generation: 0.3 seconds
- Result submission + verification: 2 seconds
- **Total:** 5.8 seconds

For long-running jobs (e.g., LLM inference > 30 seconds), overhead is negligible ($< 20\%$).

9 Security Analysis

9.1 Threat Mitigation

9.2 Known Limitations

1. **EPC Swapping:** Intel SGX’s encrypted page cache (EPC) is limited to 128 MB (256 MB on Ice Lake). Large datasets require page swapping, reducing throughput to ~ 100 MiB/s. GPU TEEs recommended for video/LLM workloads.

Threat	Mitigation
Malicious worker submits fake result	On-chain verifier rejects unless vendor quote and Merkle root match encrypted input hash. Fraud proofs slash stake.
State rollback inside enclave	Worker must report <code>prevTraceRoot</code> ; verifier checks continuity. Missing link $\Rightarrow$ slash.
Side-channel attacks (LVI, SGXaxe)	Workers run only on microcode-patched CPUs. Alternatively, job spec can request GPU-only execution.
Consensus re-org affects job state	Job receipts finalized only after 3 L1 blocks. dApps wait or re-submit.
Data unavailability	Result ciphertext stored in IPFS + optional Arweave. Merkle root on-chain ensures tamper-proof.

Table 3: Security threat model and mitigations

2. **Not Full FHE:** Operators can observe memory access patterns. For complete pattern privacy, combine with ZK-SNARKs or use fully homomorphic encryption.
3. **Attestation Licensing:** Intel DCAP quote generation incurs $\sim \$0.02/\text{quote}$. This cost is passed to job submitters.
4. **No Cross-Enclave Composability:** Currently, TEEs cannot directly call each other. Cross-TEE workflows require round-trip via L1 transactions.

10 Related Work

10.1 Confidential Computing

Secret Network [3] uses Intel SGX but relies on a trusted coordinator and single-vendor TEE. Lux supports multi-vendor TEEs and decentralized scheduling.

Oasis Sapphire [4] provides confidential EVM execution but requires all validators to run TEEs. Lux separates consensus (L1) from computation (workers), enabling heterogeneous hardware.

Flashbots SUAVE [10] uses TEEs for MEV protection but focuses on orderflow confidentiality rather than general compute.

10.2 Off-Chain Computation

iExec [5] offers decentralized cloud compute but lacks hardware confidentiality guarantees. Workers can inspect data in plaintext.

TrueBit [6] uses interactive verification games for off-chain compute correctness but has high on-chain overhead. Lux uses TEE attestation for one-shot verification.

Arbitrum Nitro [11] provides optimistic rollups with fraud proofs but no confidentiality. Lux combines fraud proofs with hardware-enforced privacy.

10.3 Multi-Party Computation

MPC protocols [7] provide cryptographic privacy without hardware trust but suffer 100-1000× performance overhead. Lux achieves near-native speed via TEEs while maintaining decentralization through attestation verification.

11 Conclusion and Future Work

We have presented Lux TEE Mesh, a production-ready decentralized confidential computing platform. The system supports heterogeneous TEE hardware, provides cryptographic attestation verification on-chain, and scales linearly with worker count. Our economic model ensures honest behavior through staking and fraud proofs.

11.1 Future Directions

- **Post-Quantum Cryptography:** Upgrade attestation signature schemes to lattice-based algorithms [12].
- **Cross-Enclave Calls:** Implement direct TEE-to-TEE secure channels using TLS with attestation-bound certificates.
- **ZK + TEE Hybrid:** Combine SNARK proofs with attestation for pattern-hiding confidential compute [13].
- **Federated Learning:** Extend to support multi-party model training with gradient aggregation in TEEs.
- **Regulatory Compliance:** Add selective disclosure primitives for GDPR/HIPAA-compliant confidential analytics.

Acknowledgments

We thank the Lux Foundation for early feedback, the Intel SGX team for DCAP integration support, and the open-source community for libp2p and containerd contributions.

References

- [1] Storm, A. et al. (2023). *Privacy on Ethereum: Current State and Future Directions*. Ethereum Foundation Research.
- [2] Rocket, S., Yin, M., Sekniqi, K., van Renesse, R., & Sirer, E. G. (2020). *Scalable and Probabilistic Leaderless BFT Consensus through Metastability*. arXiv:1906.08936.
- [3] Secret Network Foundation (2023). *Secret Network: Privacy-Preserving Smart Contracts*. <https://scrt.network/whitepaper>
- [4] Oasis Labs (2024). *Oasis Sapphire: Confidential EVM ParaTime*. <https://docs.oasis.io/sapphire>
- [5] Felson, J. et al. (2017). *iExec: Blockchain-Based Decentralized Cloud Computing*. iExec Whitepaper v3.0.

- [6] Teutsch, J., & Reitwießner, C. (2017). *A Scalable Verification Solution for Blockchains*. TrueBit Protocol.
- [7] Evans, D., Kolesnikov, V., & Rosulek, M. (2018). *A Pragmatic Introduction to Secure Multi-Party Computation*. Foundations and Trends in Privacy and Security, 2(2-3).
- [8] Vyzovitis, D. et al. (2020). *GossipSub: Attack-Resilient Message Propagation in the Filecoin and Eth2.0 Networks*. arXiv:2007.02754.
- [9] Van Bulck, J. et al. (2020). *LVI: Hijacking Transient Execution through Microarchitectural Load Value Injection*. IEEE S&P 2020.
- [10] Flashbots (2024). *SUAVE: The Single Unifying Auction for Value Expression*. <https://writings.flashbots.net/suave>
- [11] Felten, E., Kell, H., & Goldfeder, S. (2024). *Arbitrum Nitro: A Second-Generation Optimistic Rollup*. Offchain Labs Technical Report.
- [12] NIST (2024). *Post-Quantum Cryptography Standardization: Selected Algorithms*. <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [13] Chiesa, A. et al. (2023). *Combining Zero-Knowledge Proofs with Trusted Execution for Privacy and Verifiability*. IACR ePrint 2023/456.
- [14] Balduf, L. et al. (2022). *Security Analysis of GossipSub in Adversarial Networks*. ACM CCS 2022.
- [15] Yin, M. et al. (2019). *HotStuff: BFT Consensus with Linearity and Responsiveness*. ACM PODC 2019.

A SGX Quote Format Details

An SGX Quote v4 consists of the following fields (432 bytes total):

Offset	Size	Field
0	4	version (3 or 4)
4	2	attestation_key_type (ECDSA P-256)
6	4	reserved
10	32	qe_report_hash
42	32	report_data_1 (job payload hash)
74	32	report_data_2 (merkle root)
106	32	mrenclave (enclave measurement)
138	32	mrsigner (enclave signer)
170	64	reserved
234	198	signature (ECDSA-P256)

B AMD SEV-SNP Attestation Report

SEV-SNP reports (1184 bytes) include:

Offset	Size	Field
0	4	version
4	4	guest_svn
8	8	policy
16	80	family_id, image_id
96	32	measurement
128	64	host_data (job payload + merkle root)
192	32	id_key_digest
224	960	signature + certificates

C NVIDIA GPU TEE Attestation

Hopper GPU AttestationToken (512 bytes):

Offset	Size	Field
0	64	header
64	32	nonce (job payload + merkle root)
96	32	timestamp
128	32	gpu_measurement
160	352	ECDSA-P521 signature + cert chain